

Test Task — CryptoYield

Full-Stack Engineer (Senior / Lead) — невелике практичне завдання

Estimated time: 3-4 години **Stack:** FastAPI + Next.js + Postgres **Deadline:** 5 днів від отримання starter-репо

Коротко про правила гри. Ми поважаємо твій час. Це завдання навмисно *маленьке* — 3-4 години роботи. Якщо ти витратив більше 5 годин — зупинись і надішли, що є: ми хочемо побачити твій стиль, а не змарнований вечір. Ми оцінюємо **якість, а не об'єм**. Краще менше фіч і чистий код, ніж усе зроблене "на швидку".

1. Контекст

CryptoYield — crypto-платформа пасивного доходу: Next.js 14 + FastAPI + Postgres + Redis, працює у продакшні. Поточна проблема — усі юзери шлють депозити на одну спільну адресу кожної валюти, що ускладнює reconciliation. Найближча задача в беклозі — перейти на **індивідуальні depositні адреси per user**.

Тестове — це сильно спрощений прототип цієї задачі. **Реальний blockchain не використовується** — все моковано. Ми хочемо побачити, як ти думаєш про гроші, ідемпотентність і транзакційність, не витрачаючи твій час на crypto-інтеграції.

2. Що надається (starter-репо)

Ти отримаєш приватний репозиторій `cryptoyield-test-task` зі скелетом:

```
cryptoyield-test-task/
├── backend/                                # FastAPI + SQLAlchemy + Alembic
│   ├── app/
│   │   ├── main.py                        # app factory, CORS
│   │   ├── db.py                         # session, engine
│   │   ├── models.py                     # User, Deposit, DepositAddress (готово)
│   │   ├── auth.py                       # fake auth dependency (X-User-Id header)
│   │   └── routes/
│   │       └── deposits.py               # ← тут ти пишеш
│   ├── alembic/                          # міграції (initial вже є)
│   ├── tests/                            # pytest + httpx, один smoke test
│   ├── pyproject.toml
│   └── docker-compose.yml                # Postgres
├── frontend/                             # Next.js 14, App Router, TypeScript
│   ├── app/
│   │   ├── layout.tsx
│   │   └── deposit/
│   │       └── page.tsx                  # ← тут ти пишеш
│   ├── lib/api.ts                        # fetch-клієнт (готово)
│   ├── package.json
│   └── README.md                         # інструкції запуску
```

Авторизація спрощена: HTTP header `X-User-Id: <uuid>` визначає юзера. Це testing-only — у реальному кодi був би JWT. Два seed-юзери створюються автоматично (див. README).

3. Задача

Backend (FastAPI) — 3 endpoints

POST /api/deposits/address

Request: `{ "currency": "BTC" | "ETH" | "USDT" }`

Response: `{ "address": "...", "currency": "...", "created_at": "..." }`

- Повертає deposit-адресу юзера для вказаної валюти.
- Якщо в юзера вже є адреса для цієї валюти — повертає ту ж саму (не створює нову).
- Генерація адреси — mocked, за patterns: BTC: `bc1q` +32 hex; ETH: `0x` +40 hex; USDT: `T` +33 alphanum. Implementation: `app.services.address_gen` — заглушка вже в репо.

GET /api/deposits

Response: `{ "items": [...], "total": number }`

- Список депозитів поточного юзера, найновіші першими.
- Query params: `?limit=20&offset=0` (defaults: 20, 0).
- Кожен item: `id, currency, amount, tx_hash, status, created_at, confirmed_at`.

POST /api/deposits/webhook

Request: `{ "address": "...", "tx_hash": "...", "amount": "12.5", "currency": "ETH" }`

Response: `{ "status": "confirmed" | "ignored" }`

- Імітує webhook від blockchain-сканера. Викликається з Postman / curl — жодного auth header тут немає (публічний endpoint, як справжній webhook).
- Знаходить юзера за `address` → створює запис `Deposit` зі статусом `confirmed` → збільшує `user.balance_<currency>` на `amount`.
- **Критично:** повторний виклик із тим самим `tx_hash` має бути *no-op* (повертає `"ignored"`), не подвоює баланс.
- **Критично:** паралельні виклики з різними `tx_hash` для того самого юзера мають коректно сумуватися без race conditions.
- Якщо `address` невідома — `404`.

Security note. У продакшні webhook був би підписаний HMAC-секретом. Для тестового достатньо додати коментар, як би ти це зробив, — писати саму перевірку не треба.

Frontend (Next.js) — одна сторінка

Сторінка `/deposit` , доступна тільки для авторизованого юзера (юзер визначається через вибір у top-bar — fake-switcher між двома seed-юзерами; вже у starter-репо).

- Dropdown валюти (BTC / ETH / USDT) + кнопка **"Get deposit address"**.
- Після кліку — показати адресу в моноширинному шрифті з кнопкою "Copy".
- Таблиця останніх депозитів юзера: currency, amount, tx_hash (truncated), status, created_at.
- Таблиця оновлюється після нового депозиту. Polling кожні 5с або manual refresh button — на твій вибір, обґрунтуй у README.
- Мінімальний styling — Tailwind уже налаштований. Головне — **не ламається і не спамить запитами**.

4. Acceptance criteria

- ☐ `docker-compose up -d` піднімає Postgres, `pytest` проходить зелений.
- ☐ Три endpoints відповідають згідно специфікації вище.
- ☐ Повторний `POST /deposits/address` для тієї ж валюти повертає ту ж адресу (idempotent).
- ☐ Повторний webhook з тим самим `tx_hash` не подвоює баланс (idempotent).
- ☐ Є щонайменше **один тест** на idempotency webhook (пишеться той самий `tx_hash` двічі → balance змінюється один раз).
- ☐ Є **ще один тест** на твій вибір (наприклад, concurrent webhook або 404 для unknown address).
- ☐ Frontend `/deposit` рендериться, дає взяти адресу і показує таблицю.
- ☐ `npm run build / npm run lint` проходить без помилок.
- ☐ Гроші використовують `Decimal`, не `float` (і на Python-, і на TS-сторонах).

5. Tech requirements

Частина	Вимоги
Backend	Python 3.11+, FastAPI, SQLAlchemy 2.x (async або sync — на твій вибір), Pydantic v2, Alembic для міграцій, pytest. Decimal для money, не float.
Frontend	Next.js 14 App Router, TypeScript (strict), Tailwind. Server Components чи Client Components — на твій вибір, обґрунтуй у README одним реченням.
DB	Postgres. Міграція в Alembic. Якщо додаєш нові поля / таблиці — не редагуй існуючу migration, роби <code>alembic revision</code> .
Redis	Не потрібен. Якщо дуже хочеш — можеш використати для idempotency, але Postgres-unique теж ок.
Code style	ruff + black (config у репо) на Python, eslint + prettier на TS. Не треба нових правил, достатньо проходити існуючі.

6. Bonus — архітектурні питання (optional)

У кінці твого README додай короткі відповіді (2-4 речення на кожне). Це необов'язково, але сильно впливає на нашу оцінку. Якщо не встигаєш — пропусти, ми запитаємо в інтерв'ю.

- Idempotency:** як саме ти гарантував, що повторний webhook не подвоїть баланс? Які ще способи є? Чому обрав цей?
- Race conditions:** припусти, два webhooks для того ж юзера прилітають одночасно. Чому твій код не ламається? Покажи конкретний рядок / механізм.

- 3. **Production hardening:** три речі, які ти б зробив перш ніж пускати цей код у прод (HMAC webhook? rate limiting? alerts? щось своє?).
- 4. **HD wallets:** як би ти переписав address-generation, щоб адреси були *реальні*, з master-seed, deterministic per user? Де зберігав би seed? Одного абзацу достатньо.

7. Submission

- 1. Працюй у своєму fork'у репозиторія або на гілці `candidate/oleksandr` (на твій вибір).
- 2. Коміти — бажано атомарні, з conventional-commits style (`feat:` , `fix:` , `test:`).
- 3. Відкрий **Pull Request** у main — ми рев'юватимемо в PR-форматі.
- 4. У description PR коротко: що зробив, чого не встиг, будь-які трейд-офи, на які хочеш звернути увагу.
- 5. Deadline: **5 днів** від отримання репо. Якщо треба більше — напиши, домовимося.

8. Критерії оцінки (чесно і прозоро)

Що дивимось	Вага
Коректність грошових операцій. Idempotency webhook, race safety, Decimal. Головне.	40%
Code quality. Типізація, error handling, читабельність, розумний розмір функцій, відсутність dead code.	25%
API design. Коди відповідей, schema, назви полів, ідемпотентність address endpoint.	15%
Frontend. Що працює без багів, не спамить запитами, UX адекватний для крипто (copy address тощо).	10%
Bonus-питання + README. Ясність пояснень, trade-offs.	10%

Що не оцінюємо: красу UI, CSS-анімації, 100% test coverage, AWS-інтеграції, Docker-оптимізації, монорепо-tooling, мобільний responsive, i18n.

9. Наступні кроки

- 1. Ти надсилаєш PR.
- 2. Ми рев'юємо — 1-2 робочі дні.
- 3. Якщо добре — **30-хв розмова** по твоєму коду: пройдемося по твоїх рішеннях, обговоримо що б ти покращив.
- 4. Якщо все ок — **paid trial day** на реальну задачу з проекту, потім оффер.

Питання про завдання — пиши в Telegram / email. Без вагань: краще уточнити score за 5 хв, ніж написати зайве.